

03 API - Relaties tussen entiteiten

📅 09-09-2026 ⌚ Ongeveer 22 minuten 🗨️ Ongeveer 3256 woorden

In dit hoofdstuk leer je hoe je relaties legt tussen entiteiten via het code-first principe. Daarna wordt uitgelegd hoe je gegevens uit meerdere entiteiten ophaalt, toevoegt, wijzigt en verwijdert met een API.

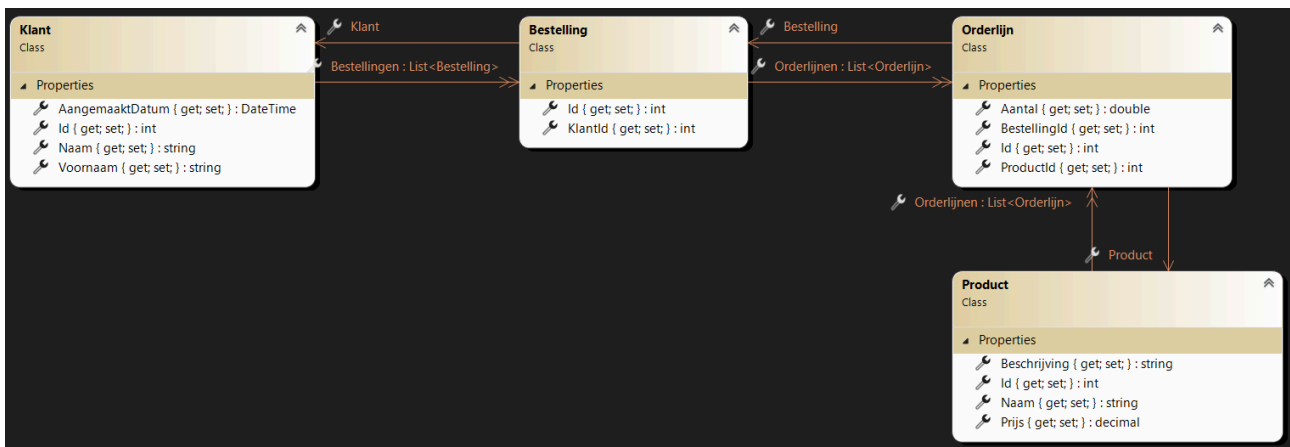
Setup

> [Clone de voorbeeldRepo voor dit hoofdstuk.](#)

In dit project zijn een aantal extra modellen toegevoegd.

Navigation properties

In de cursus van Programming Advanced is dit hoofdstuk al aan bod gekomen. Navigation properties zorgen ervoor dat je een route legt van de ene entiteit naar een andere zodat je gegevens over die entiteit kunt ophalen. Hieronder vind je het klassediagram voor dit hoofdstuk:



Relatiebeschrijvingen:

- Een klant kan meerdere bestellingen aanmaken, een bestelling hoort bij één klant. De Foreign Key staat in de tabel `Bestelling`. Dit is een één-op-veel relatie.
- Op een bestelling kun je meerdere producten bestellen en een product kan voorkomen op meerdere bestellingen. De Foreign Key's voor Product en Bestelling staan in de associatie-entiteit `Orderlijn`. Dit is een veel-op-veel relatie.

Als je de navigatieproperties toevoegt, volgens de relatiebeschrijvingen, krijg je:

- Een klant heeft een lijst van bestellingen

```
1 public class Klant
2 {
3     //overige attributen
4     public List<Bestelling> Bestellingen { get; set; }
5 }
```

- Een bestelling behoort bij één klant (hier komt de FK van klant) en een bestelling heeft een lijst van orderlijnen. Je voegt hier een property toe van het type Klant om de klantgegevens te kunnen ophalen.

```
1 public class Bestelling
2 {
3     public int Id { get; set; }
4
5     public int KlantId { get; set; }
6
7     public Klant Klant { get; set; } = default!;
8
9     public List<Orderlijn> Orderlijnen { get; set; } = default!;
10 }
```

- Een product komt voor op meerdere orderlijnen.

```
1 public class Product
2 {
3     //overige attributen
4     public List<Orderlijn> Orderlijnen { get; set; }
5 }
```

- Een orderlijn hoort bij één product (FK) en bij één bestelling (FK). Je voegt ook een attribuut Bestelling en een attribuut Product om gegevens van die objecten te kunnen ophalen.

```
1 public class Orderlijn
2 {
3     // Overige attributen
4     public int BestellingId { get; set; }
```

```

5     public Bestelling Bestelling { get; set; }
6
7     public int ProductId { get; set; }
8
9     public Product Product { get; set; }
10 }
11

```

Fluent API

Je kunt op dit moment een migratie uitvoeren en dan zal de relatie in de database worden aangemaakt. Als je geen delete behaviour opgeeft, zal, afhankelijk van het type relatie en of de foreign key nullable is, het delete behaviour ingesteld worden als **Cascade** of **ClientSetNull**. Als de FK niet-nullable is, is het standaard delete behaviour **Cascade**. Indien de Foreign Key wel optioneel is, wordt het standaard delete behaviour **ClientSetNull**. Bestudeer zeker het onderstaande voorbeeld:

```

1 // Required relatie - standaard Cascade
2 modelBuilder.Entity<Order>()
3     .HasOne(o => o.Customer)
4     .WithMany(c => c.Orders)
5     .HasForeignKey(o => o.CustomerId); // CustomerId is int (non-
6 nullable)
7
8 // Optionele relatie - standaard ClientSetNull
9 modelBuilder.Entity<Order>()
10     .HasOne(o => o.Customer)
11     .WithMany(c => c.Orders)
12     .HasForeignKey(o => o.CustomerId); // CustomerId is int? (nullable)

```

Let op: **ClientSetNull** werkt alleen als de gerelateerde entiteiten geladen zijn in de DbContext. Voor meer voorspelbare resultaten kun je beter expliciet **DeleteBehavior.SetNull** gebruiken voor optionele relaties.

Het is efficiënter om de relaties en het delete behaviour expliciet te definiëren in je databasecontext bij de methode **OnModelCreating**.

```

1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     base.OnModelCreating(modelBuilder);
4     modelBuilder.Entity<Klant>().ToTable("Klant");
5     modelBuilder.Entity<Bestelling>().ToTable("Bestelling");
6 }

```

```

5     modelBuilder.Entity<Orderlijn>().ToTable("Orderlijn");
6     modelBuilder.Entity<Product>
7     ().ToTable("Product").Property(p=>p.Prijs).HasColumnType("decimal(18,2)");
8
9     // One to many
10    modelBuilder.Entity<Bestelling>()
11        .HasOne(p => p.Klant)
12        .WithMany(x => x.Bestellingen)
13        .HasForeignKey(y => y.KlantId)
14        .OnDelete(DeleteBehavior.Restrict)
15        .IsRequired();
16
17    // Many to many
18    modelBuilder.Entity<Orderlijn>()
19        .HasOne(p => p.Bestelling)
20        .WithMany(x => x.Orderlijnen)
21        .HasForeignKey(y => y.BestellingId)
22        .OnDelete(DeleteBehavior.Restrict)
23        .IsRequired();
24
25    modelBuilder.Entity<Orderlijn>()
26        .HasOne(p => p.Product)
27        .WithMany(x => x.Orderlijnen)
28        .HasForeignKey(y => y.ProductId)
29        .OnDelete(DeleteBehavior.Restrict)
30        .IsRequired();
31 }

```

Het principe is als volgt: je beschrijft elke "veel-kant" van een relatie. Bv Een klant kan meerdere bestellingen doen. De relatie wordt dus beschreven op basis van de entiteit `Bestelling`. De één-kant van `Klant` wordt niet beschreven.

- `.HasOne(p => p.Klant)`: Elke bestelling heeft één klant
- `.WithMany(x => x.Bestellingen)`: Een klant heeft meerdere bestellingen
- `.HasForeignKey(y => y.KlantId)`: De foreign key is `KlantId`
- `.OnDelete(DeleteBehavior.Restrict)`: Restrict: Als je een klant verwijdert, moet je eerst de bestellingen verwijderen van die klant.
- `.IsRequired()`: Elke bestelling moet behoren tot een klant.

In `Orderlijn` komen 2 "veel-kanten" toe, dus beschrijf je ze allebei in je databasecontext.

Het beschrijven van de relaties in `OnModelCreating` is een voorbeeld van **Fluent API**. Fluent API biedt dezelfde mogelijkheden als data-annotations maar heeft ook nog extra mogelijkheden zoals het wijzigen van de tabelnaam, kolomnaam, relatiebeschrijvingen of het samenstellen van complexe sleutels. In Fluent API gebruik je **method chaining** voor het koppelen van meerdere methodes aan elkaar. Hieronder vind je een voorbeeld van **method chaining**:

```
1 modelBuilder.Entity<Product>()  
2     .Property(p => p.Name)  
3     .IsRequired() // Verplicht veld  
4     .HasMaxLength(50); // Maximale lengte van 50 tekens
```

CS

Verwijder je database `WebApiDemo`. Maak een nieuwe migratie aan en update de database. Voer het script `Data\Script\WebApiDemo.sql` uit.

API - gegevens ophalen uit meerdere entiteiten

Als je in een SQL-statement gegevens wil ophalen uit meerdere tabellen, maak je gebruik van Joins (Inner, Outer, Left, Right). In Entity Framework worden geen SQL-statementen gebruikt, maar LINQ. In LINQ gebruik je de methodes `Include` en `ThenInclude` om gegevens uit meerdere tabellen op te halen. Je combineert meerdere dbsets met behulp van **method chaining**. Hierbij moet je altijd goed het klassendiagram of ERD beschouwen. Volgende voorbeelden zijn gebaseerd op het bovenstaande klassendiagram:

Het principe is als volgt: de dbsets die rechtstreeks verbonden zijn met de dbset, waarvan je vertrekt, verbind je met de methode `.Include`. vb. Je geeft de dbset `Bestellingen` mee als argument in de lambda-expressie van de `.Include`.

LINQ

```
1 // GET: api/Klant  
2 [HttpGet]  
3 public async Task<ActionResult<IEnumerable<Klant>>>  
4 AlleKlantenMetBestellingenOphalen()  
5 {  
6     return await  
7     _context.Klanten.Include(x=>x.Bestellingen).ToListAsync();  
8 }
```

CS

SQL

```

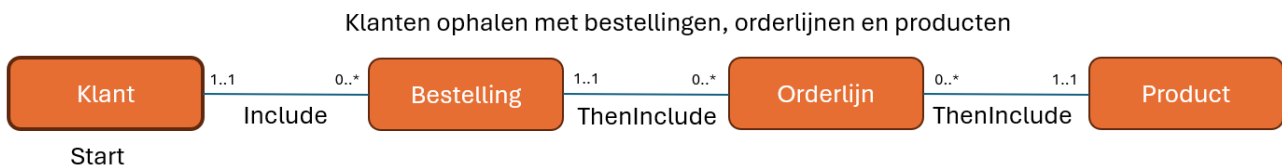
1  var rows = await _context.Set<KlantBestellingJoin>()
2      .FromSqlRaw(@"
3      SELECT
4          k.Id AS KlantId, k.Naam AS KlantNaam,
5          b.Id AS BestellingId, b.KlantId AS BestellingKlantId
6      FROM Klanten k
7      LEFT JOIN Bestellingen b ON b.KlantId = k.Id
8      ")
9      .ToListAsync();

```

Omdat je hier alleen Klant → Bestellingen nodig hebt, kun je één SQL-query met een LEFT JOIN gebruiken en daarna in C# groeperen

Orderlijnen en Producten bevinden zich, in het klassendiagram, 'achter' de dbset Bestellingen, ten opzichte van de dbset Klanten.

Deze entiteiten koppel je met een `.ThenInclude`.



LINQ

```

1  // GET: api/Klant
2  [HttpGet]
3  public async Task<ActionResult<IEnumerable<Klant>>> AlleKlantenMetBestell
4  {
5      return await
6      _context.Klanten.Include(x=>x.Bestellingen).ThenInclude(x=>x.Orderlijnen)
7  }

```

SQL

```

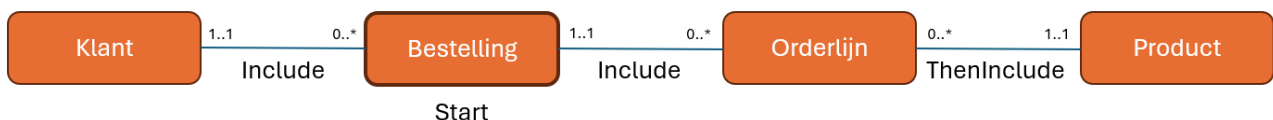
1  var rows = await _context.Set<KlantBestellingJoin>()
2      .FromSqlRaw(@"
3      SELECT
4          k.Id AS KlantId, k.Naam AS KlantNaam,
5          b.Id AS BestellingId, b.KlantId AS BestellingKlantId,
6          ol.BestellingId AS OrderlijnBestellingId, ol.ProductId AS
7      OrderlijnProductId,
8          p.Id AS ProductId, p.Naam AS ProductNaam, p.Prijs AS
9      ProductPrijs
10     FROM Klanten k
11     LEFT JOIN Bestellingen b ON b.KlantId = k.Id
12     LEFT JOIN Orderlijnen ol ON ol.BestellingId = b.Id
13     LEFT JOIN Producten p ON ol.ProductId = p.Id
        ")
        .ToListAsync();

```

Het is mogelijk om alles in één SQL-query op te halen met JOINS, zodat je Klanten, hun Bestellingen, en de Orderlijnen met Product in één keer krijgt. Het nadeel is dat de resultaten gedupliceerd worden (elke combinatie van Klant x Bestelling x Orderlijn x Product is één rij), dus je moet in C# daarna de objecten groeperen en mappen.

In het volgende voorbeeld, vertrekt de relatie vanuit Bestelling. Je wil een lijst van bestellingen en je wil gegevens van de klanten en producten toevoegen. De dbsets `Klanten` en `Orderlijnen` zijn nu rechtstreeks verbonden met `Bestellingen`. Dus gebruik je daar een `.Include`. Enkel voor de dbset `Product`, die achter `Orderlijnen` ligt, gebruik je een `.ThenInclude`.

Bestellingen ophalen met klantgegevens, orderlijnen en producten



LINQ

```

1  // GET: api/Bestelling
2  [HttpGet]
3  public async Task<ActionResult<IEnumerable<Bestelling>>> AlleBestellingen
4  {
5      return await

```

```
6     _context.Bestellingen.Include(x=>x.Klant).Include(x=>x.Orderlijnen).ThenI
    }
```

SQL

```
1      var rows = await _context.Set<BestellingProductJoin>()  
2          .FromSqlRaw(@"  
3              SELECT  
4                  b.Id AS BestellingId, b.KlantId,  
5                  k.Id AS KlantId, k.Naam AS KlantNaam,  
6                  ol.BestellingId AS OrderlijnBestellingId, ol.ProductId  
7      AS OrderlijnProductId,  
8                  p.Id AS ProductId, p.Naam AS ProductNaam, p.Prijs AS  
9      ProductPrijs  
10         FROM Bestellingen b  
11         INNER JOIN Klanten k ON b.KlantId = k.Id  
12         LEFT JOIN Orderlijnen ol ON b.Id = ol.BestellingId  
13         LEFT JOIN Producten p ON ol.ProductId = p.Id  
14     ")  
15     .ToListAsync();
```

Als je gaat werken met Repositories, vervang je `.Include` en `.ThenInclude` door respectievelijk `.Select` en `.ThenSelect`. Dit leer je in het volgende hoofdstuk.

JsonIgnore en ReferenceLoops

Als je de bovenstaande actie aanspreekt, stuit je echter op een probleem. Dit is te wijten aan de navigation properties.

An unhandled exception occurred while processing the request.

```

JsonException: A possible object cycle was detected. This can either be due to a cycle or if the object depth
ReferenceHandler.Preserve on JsonSerializerOptions to support cycles. Path:
$.klant.bestelling.klant.bestelling.klant.bestelling.klant.bestelling.klant.bestelling.klant.bestelling.klant.bestellin
System.Text.Json.ThrowHelper.ThrowJsonException(SerializerCycleDetected(int maxDepth)

```

De oorzaak is een oneindige lus. Een bestelling heeft een klant, een klant heeft bestellingen, bestellingen hebben klanten ...

Je kunt dit voorkomen door in `Program.cs` in te stellen dat er in Json geen loops mogen voorkomen:

```
1 builder.Services.AddControllers().AddJsonOptions(options => options.JsonSerializerOptions.ReferenceHandler=ReferenceHandler.IgnoreCycle
```

Voeg ook de juiste referentie toe in `GlobalUsings.cs` .

```
1 global using System.Text.Json.Serialization;
```

Als je de bestellingen nu opvraagt, krijg je volgend resultaat:

```
[
  {
    "bestellingID": 1,
    "klantID": 1,
    "klant": {
      "klantID": 1,
      "naam": "Van Der Neffe",
      "voornaam": "Leon",
      "aangemaaktDatum": "2022-10-01T00:00:00"
    },
    "orderlijnen": [
      {
        "orderLijnID": 1,
        "aantal": 3,
        "bestellingID": 1,
        "productID": 1,
        "product": {
          "productID": 1,
          "naam": "fiets",
          "prijs": 100,
          "merk": "Gazelle",
          "beschrijving": "Dit is een fiets",
          "orderlijnen": [
            {
              "orderLijnID": 3,
              "aantal": 4,
              "bestellingID": 2,
              "bestelling": {
                "bestellingID": 2,
                "klantID": 2,
                "klant": {
                  "klantID": 2,
                  "naam": "Van De Kasseinen",
                  "voornaam": "Firmin",
                  "aangemaaktDatum": "2022-10-02T00:00:00"
                }
              }
            }
          ]
        }
      }
    ]
  }
]
```

Bij de orderlijnen worden de producten opgehaald, maar bij de producten worden ook de orderlijnen opgehaald met daarbij de bestelling en de klant. Om te zorgen dat er bepaalde navigation properties niet worden opgehaald, kun je het kenmerk **[JsonIgnore]** gebruiken bij bepaalde properties in je model. Deze properties worden dan niet getoond in je Json-resultaat.

```
1 [JsonIgnore]
2 public List<Orderlijn> Orderlijnen { get; set; } = default!;
```

CS

API - Ingeven van entiteiten met relaties

In het onderstaande voorbeeld wordt een bestelling ingegeven, met klant en orderlijnen:

LINQ

```
1  // POST: api/Bestelling
2  [HttpPost]
3  public async Task<ActionResult<Bestelling>>
4  BestellingToevoegen(Bestelling bestelling)
5  {
6      if (_context.Bestellingen == null)
7          return NotFound("De tabel Bestellingen bestaat niet in de
8  database.");
9      if (!ModelState.IsValid)
10     {
11         return BadRequest(ModelState);
12     }
13
14     _context.Bestellingen.Add(bestelling);
15     try
16     {
17         await _context.SaveChangesAsync();
18     }
19     catch (DbUpdateException dbError)
20     {
21         return BadRequest(dbError);
22     }
23
24     return CreatedAtAction("BestellingOphalen", new { id =
25     bestelling.Id }, bestelling);
26 }
```

CS

SQL

```
1 [HttpPost]
2 public async Task<ActionResult<Bestelling>>
3 BestellingToevoegen(Bestelling bestelling)
4 {
5     if (_context.Bestellingen == null)
6         return NotFound("De tabel Bestellingen bestaat niet in de
7 database.");
8
9     if (!ModelState.IsValid)
10        return BadRequest(ModelState);
11
12    using var transaction = await
13 _context.Database.BeginTransactionAsync();
14
15    try
16    {
17        // Voeg de bestelling toe
18        await _context.Database.ExecuteSqlRawAsync(
19            "INSERT INTO Bestellingen (KlantId) VALUES ({0})",
20            bestelling.KlantId
21        );
22
23        // Haal het Id van de nieuwe bestelling
24        var nieuwBestellingId = await _context.Bestellingen
25            .FromSqlRaw("SELECT TOP(1) * FROM Bestellingen ORDER BY Id
26 DESC")
27            .Select(b => b.Id)
28            .FirstOrDefaultAsync();
29
30        bestelling.Id = nieuwBestellingId;
31
32        // Voeg alle Orderlijnen toe
33        foreach (var ol in bestelling.Orderlijnen)
34        {
35            await _context.Database.ExecuteSqlRawAsync(
36                "INSERT INTO Orderlijnen (BestellingId, ProductId)
37 VALUES ({0}, {1})",
38                bestelling.Id,
39                ol.ProductId
40            );
41        }
42
43        // Commit de transactie
```

CS

```
43         await transaction.CommitAsync();
44     }
45     catch (DbUpdateException dbError)
46     {
47         await transaction.RollbackAsync();
48         return BadRequest(dbError);
49     }
50
51     return CreatedAtAction("BestellingOphalen", new { id =
bestelling.Id }, bestelling);
52 }
```

Je ziet dat de achterliggende code niet verschilt van het ingeven van één product of klant. Dit is dankzij een ingebouwd mechanisme van het Entity Framework: Eerst voeg je een nieuwe Bestelling-entiteit toe aan de context. Dit betekent dat Entity Framework (EF) nu weet dat er een nieuwe Bestelling-entiteit is die moet worden toegevoegd aan de database.

```
1  _context.Bestellingen.Add(bestelling);
```

CS

Daarna voer je alle wijzigingen die tot nu toe zijn toegevoegd aan de context door naar de database. In dit geval wordt de nieuwe Bestelling-entiteit toegevoegd aan de Bestellingen-tabel in de database. Zodra dit gebeurt, wordt de gegenereerde Id voor deze Bestelling automatisch bijgewerkt in het bestelling object dankzij EF's identity mechanisme. Dit betekent dat er in de orderlijnen het bestellingId wordt toegevoegd. Met `SaveChangesAsync()` of `SaveChanges()` worden ook de orderlijnen toegevoegd aan de database.

```
1  await _context.SaveChangesAsync();
```

CS

Problemen:

- Als je een nieuwe bestelling aanmaakt, verwacht de API ook een nieuwe klant.
- In de orderlijnen verwacht de API ook nieuwe bestellingen en nieuwe producten.

Om dit op te lossen, maak je de navigation properties optioneel met het **vraagteken** en negeer je het tonen van de navigation properties met het kenmerk **[JsonIgnore]**.

```
1 public int ProductId { get; set; }
2 [JsonIgnore]
3 public Product? Product { get; set; } = default!;
4 [JsonIgnore]
5 public Bestelling? Bestelling { get; set; } = default!;
```

CS

Je geeft bij het uitvoeren van je API enkel de KlantId, Aantal en ProductId in. De BestellingId wordt automatisch ingevuld. In een later hoofdstuk, leer je hoe je met het behulp van een DTO (Data Transfer Object) enkel de benodigde gegevens kunt tonen.

POST /api/Bestelling

Parameters Cancel Reset

No parameters

Request body application/json

```
{
  "id": 0,
  "klantId": 1,
  "orderlijnen": [
    {
      "id": 0,
      "aantal": 10,
      "bestellingId": 0,
      "productId": 1
    },
    {
      "id": 0,
      "aantal": 20,
      "bestellingId": 0,
      "productId": 2
    }
  ]
}
```

Execute

API - gegevens wijzigen met relaties

Het wijzigen van gegevens met relaties tussen entiteiten is een complex gegeven. Je moet hiervoor rekening houden met volgende stappen:

1. Controleer of de id's ondertussen niet gewijzigd zijn.
2. Haal de oorspronkelijke gegevens op, met de onderliggende associaties en controleer of er een record gevonden is.
3. Valideer of de nieuwe gegevens correct zijn.
4. Wijzig het record met de nieuwe gegevens.
5. Voor een associatie wijzig je bestaande records, voeg je nieuwe records toe en verwijder je overbodige records.
6. Controleer of je de wijzigingen kunt doorvoeren naar de database.

LINQ

```
1      [HttpPut("{id}")]                                CS
2      public async Task<IActionResult> BestellingWijzigen(int id, Bestelling
3      bestelling)
4      {
5          if (id != bestelling.Id)
6          {
7              return BadRequest("De id's van de bestelling die je wil
8      wijzigen komen niet overeen.");
9          }
10         var existingBestelling = await _context.Bestellingen.Include(b =>
11         b.Orderlijnen).FirstOrDefaultAsync(b => b.Id == id);
12
13         if (existingBestelling == null)
14         {
15             return NotFound("De bestelling die je wil wijzigen, komt niet
16     voor in de database.");
17         }
18
19         if (!ModelState.IsValid)
20         {
21             return BadRequest(ModelState);
22         }
23
24         existingBestelling.KlantId = bestelling.KlantId;
25
26         foreach (var updatedOrderlijn in bestelling.Orderlijnen)
27         {
28             var existingOrderlijn =
29     existingBestelling.Orderlijnen.FirstOrDefault(ol => ol.Id ==
30     updatedOrderlijn.Id);
```

```
31         if (existingOrderlijn != null)
32         {
33             existingOrderlijn.Aantal = updatedOrderlijn.Aantal;
34             existingOrderlijn.ProductId = updatedOrderlijn.ProductId;
35         }
36         else
37         {
38             existingBestelling.Orderlijnen.Add(new Orderlijn
39             {
40                 Aantal = updatedOrderlijn.Aantal,
41                 BestellingId = existingBestelling.Id,
42                 ProductId = updatedOrderlijn.ProductId
43             });
44         }
45     }
46     var orderlijnenToRemove = existingBestelling.Orderlijnen
47         .Where(ol => !bestelling.Orderlijnen.Any(uol => uol.Id ==
48 ol.Id))
49         .ToList();
50
51     _context.Orderlijnen.RemoveRange(orderlijnenToRemove);
52
53     try
54     {
55         await _context.SaveChangesAsync();
56     }
57     catch (DbUpdateConcurrencyException)
58     {
59         {
60             if (!BestellingExists(id))
61             {
62                 return NotFound("Er is een probleem opgetreden bij het
63 opslaan in de database.");
64             }
65             else
66             {
67                 throw;
68             }
69         }
70
71         return Ok($"De bestelling is gewijzigd.");
72     }
```

SQL

```
1 [HttpPut("{id}")]
2 public async Task<IActionResult> BestellingWijzigen(int id, Bestelling
3 bestelling)
4 {
5     if (id != bestelling.Id)
6     {
7         return BadRequest("De id's van de bestelling die je wil
8 wijzigen komen niet overeen.");
9     }
10
11     if (!ModelState.IsValid)
12     {
13         return BadRequest(ModelState);
14     }
15
16     using var transaction = await
17 _context.Database.BeginTransactionAsync();
18
19     try
20     {
21         // 1 Controleer of de bestelling bestaat
22         var bestaat = await _context.Bestellingen
23             .FromSqlRaw("SELECT * FROM Bestellingen WHERE Id = {0}",
24 id)
25             .AnyAsync();
26
27         if (!bestaat)
28         {
29             return NotFound("De bestelling die je wil wijzigen, komt
30 niet voor in de database.");
31         }
32
33         // 2 Update de Bestelling zelf
34         await _context.Database.ExecuteSqlRawAsync(
35             "UPDATE Bestellingen SET KlantId = {0} WHERE Id = {1}",
36             bestelling.KlantId, id
37         );
38
39         // 3 Haal bestaande orderlijnen
40         var existingOrderlijnen = await _context.Orderlijnen
```

CS

```
41         .FromSqlRaw("SELECT * FROM Orderlijnen WHERE BestellingId =
42 {0}", id)
43         .ToListAsync();
44
45     // 4 Verwerk updates en toevoegingen
46     foreach (var updatedOrderlijn in bestelling.Orderlijnen)
47     {
48         var existing = existingOrderlijnen.FirstOrDefault(ol =>
49 ol.Id == updatedOrderlijn.Id);
50         if (existing != null)
51         {
52             // Update bestaande orderlijn
53             await _context.Database.ExecuteSqlRawAsync(
54                 "UPDATE Orderlijnen SET Aantal = {0}, ProductId =
55 {1} WHERE Id = {2}",
56                 updatedOrderlijn.Aantal,
57                 updatedOrderlijn.ProductId, existing.Id
58             );
59         }
60         else
61         {
62             // Voeg nieuwe orderlijn toe
63             await _context.Database.ExecuteSqlRawAsync(
64                 "INSERT INTO Orderlijnen (BestellingId, ProductId,
65 Aantal) VALUES ({0}, {1}, {2})",
66                 id, updatedOrderlijn.ProductId,
67                 updatedOrderlijn.Aantal
68             );
69         }
70     }
71
72     // 5 Verwijder orderlijnen die niet meer aanwezig zijn
73     var orderlijnenToRemove = existingOrderlijnen
74         .Where(ol => !bestelling.Orderlijnen.Any(uol => uol.Id ==
75 ol.Id))
76         .ToList();
77
78     foreach (var ol in orderlijnenToRemove)
79     {
80         await _context.Database.ExecuteSqlRawAsync(
81             "DELETE FROM Orderlijnen WHERE Id = {0}", ol.Id
82         );
83     }
```

```
        await transaction.CommitAsync();
    }
    catch (DbUpdateException dbError)
    {
        await transaction.RollbackAsync();
        return BadRequest(dbError);
    }

    return Ok("De bestelling is gewijzigd.");
}
```

Voorbeeld verwijderen orderlijnen

```
1  // Selecteer de orderlijnen waarvoor er géén overeenkomstig Id in de      CS
2  nieuwe lijst bestaat.
3  var orderlijnenToRemove = existingBestelling.Orderlijnen
4      .Where(ol => !bestelling.Orderlijnen.Any(uol => uol.Id == ol.Id))
5      .ToList();
6
7  // voorbeeld bestaande orderlijnen
8  existingBestelling.Orderlijnen = [
9      { Id = 1, Product = "Boek" },
10     { Id = 2, Product = "Pen" },
11     { Id = 3, Product = "Tas" }
12 ]
13
14 // voorbeeld nieuwe orderlijnen
15 bestelling.Orderlijnen = [
16     { Id = 1, Product = "Boek" },
17     { Id = 3, Product = "Tas" },
18     { Id = 4, Product = "Potlood" }
19 ]
```

- Id=1 → komt nog voor → blijft
- Id=2 → komt niet meer voor → wordt verwijderd
- Id=3 → komt nog voor → blijft
- Id=4 → is nieuw → is al toegevoegd

```
1 // inhoud van OrderlijnenToRemove:
2 orderlijnenToRemove = [{ Id = 2, Product = "Pen" }]
3
4 // met dit commando verwijder je alle orderlijnen die in
5 orderlijnenToRemove zitten
_context.Orderlijnen.RemoveRange(orderlijnenToRemove);
```

CS

API - gegevens verwijderen met relaties

Het principe voor het verwijderen van gegevens met onderliggende records in een associatie-entiteit is als volgt:

1. Haal het huidige record op en controleer of er een record gevonden is.
2. Verwijder **eerst** de onderliggende records in een associatie-entiteit. In een relatie met **Cascade** is dat niet nodig, maar het is best practices om consequent dezelfde methode toe te passen.
3. Verwijder daarna het record uit de dbset en probeer de wijzigingen door te voeren naar de database.

LINQ

```
1 // DELETE: api/Bestelling/5
2 [HttpDelete("{id}")]
3 public async Task<IActionResult> BestellingVerwijderen(int id)
4 {
5     var bestelling = await _context.Bestellingen.Include(b =>
6     b.Orderlijnen).FirstOrDefaultAsync(b => b.Id == id);
7
8     if (bestelling == null)
9     {
10         return NotFound("De bestelling werd niet gevonden.");
11     }
12
13     foreach (var orderlijn in bestelling.Orderlijnen)
14     {
15         _context.Orderlijnen.Remove(orderlijn);
16     }
17
18     _context.Bestellingen.Remove(bestelling);
19 }
```

CS

```

20     try
21     {
22         await _context.SaveChangesAsync();
23     }
24     catch (DbUpdateException ex)
25     {
26         // Log de fout of geef een aangepaste foutmelding terug
27         // Algemene vorm:
28         // return StatusCode(StatusCode.Status500InternalServerError,
29 "Er is een fout opgetreden bij het verwijderen van de bestelling.");
30         // Verkorte vorm:
31         return StatusCode(500, "Er is een fout opgetreden bij het
32 verwijderen van de bestelling.");
33     }

    return Ok($"De bestelling met id {id} is verwijderd.");
}

```

SQL

```

1  [HttpDelete("{id}")]
2  public async Task<IActionResult> BestellingVerwijderen(int id)
3  {
4      using var transaction = await
5      _context.Database.BeginTransactionAsync();
6
7      try
8      {
9          // Controleer of de bestelling bestaat
10         var bestaat = await _context.Bestellingen
11         .FromSqlRaw("SELECT * FROM Bestellingen WHERE Id = {0}",
12 id)
13         .AnyAsync();
14
15         if (!bestaat)
16         {
17             return NotFound("De bestelling werd niet gevonden.");
18         }
19
20         // Verwijder alle gekoppelde orderlijnen
21         await _context.Database.ExecuteSqlRawAsync(
22             "DELETE FROM Orderlijnen WHERE BestellingId = {0}", id

```

CS

```
23         );
24
25         // Verwijder de bestelling zelf
26         await _context.Database.ExecuteSqlRawAsync(
27             "DELETE FROM Bestellingen WHERE Id = {0}", id
28         );
29
30         await transaction.CommitAsync();
31     }
32     catch (DbUpdateException)
33     {
34         await transaction.RollbackAsync();
35         return StatusCode(500, "Er is een fout opgetreden bij het
36 verwijderen van de bestelling.");
37     }
38
39     return Ok($"De bestelling met id {id} is verwijderd.");
40 }
```

Raw Queries - Waakzaamheid security

Als een query letterlijk wordt geschreven via een Raw SQL-statement, moet je altijd alert zijn om deze heel goed te beveiligen tegen injecties. Standaard beveiligt een framework bv. Entity Framework (ASP.NET) of Laravel Framework (PHP) dit, **maar** als je zelf afwijkt - bv. Raw Queries zijn daar een voorbeeld van, valt deze beveiliging vaak deels of volledig weg.

In het onderstaand voorbeeld zie je {0} als parameter in combinatie met de bijhorende opvuller ervan, dit helpt bij de filtering. Dit zorgt ervoor dat de rawquery niet geïnjecteerd kan worden met kwaardige (SQL) code door een hacker.

⚠ Let op: Gebruik `FromSqlRaw` met **parameterbinding** om SQL-injectie te vermijden. Deze methode wordt op internet ook wel een Parameterized Query genoemd.

```
1 [HttpPut("{id}")]
2 public async Task<IActionResult> BestellingWijzigen(int id, Bestelling
3 bestelling)
4 {
5     //...
6     //...
7     // Via {0}
```

CS

```

8         var existingOrderlijnen = await _context.Orderlijnen
9             .FromSqlRaw("SELECT * FROM Orderlijnen WHERE BestellingId = {0}",
10 id) // <== Parameterbinding tussen {0} en id
            .ToListAsync();
    }

```

De tip is als je geen Raw Query nodig hebt, gebruik je gewoon best het framework zijn query builder. Gebruik Raw Queries vooral wanneer je de query niet kan maken (complexere queries, specifieke subqueries) via ORM builder van bv. EF.

SQL-injectie voorbeeld

De volgende code bevat meerdere kwetsbaarheden: de id is een string en bij de query worden geen parameteriseerde beveiliging toegepast. Als de id numeriek is en hierop zou gecontroleerd worden door if/else of filterwaarde, zal de injectie hierop vastlopen.

We noemen dit voorbeeld - ⚠ letterlijke invoer van {iets} in query

```

1 public async Task<IActionResult> Index(string id)
2 {
3     var orderlijnen = await _context.Orderlijnen
4         .FromSqlRaw($"SELECT * FROM Orderlijnen WHERE BestellingId =
5 {id}")
6         .ToListAsync();
7
8     return View(orderlijnen);
9 }

```

De kwetsbaarheid in deze raw query is het letterlijk invoegen van de waarde. Deze parameter komt bv uit de GET-parameter van de url `https://localhost:1234/Bestelling?BestellingId=1`. Een hacker kan dan het verkeer omleiden en een bepaalde proxy plaatsen tussen de webapplicatie en zijn machine om manueel de GET-operaties te manipuleren met gevaarlijke datastrings. Stel dat een kwaadwillige de BestellingId wijzigt naar:

```

1 1; DROP TABLE Klant;

```

Dan wordt de Raw Query:

```

1 SELECT * FROM Orderlijnen WHERE BestellingId = 1; DROP TABLE Klant;

```

Deze verwijdert de tabel Klant en beschadigt zo de database, want dit is een kernentiteit. Uiteraard kan er nog groter schade worden aangericht. Een ander voorbeeld is het toevoegen van een extra admin account met bepaalde rechten.

Bescherm dus altijd je code!

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM